

PROJECT NAME: A Centralized Platform for Educational and Career Opportunities

GROUP MEMBERS: Kaan DÖNMEZ, Hilal Seray ORTAN, Baran Deniz AYDIN, Salih Batuhan ŞENER, Doruk YAŞAR, Selin TAY, Ömer ŞEN

TASK #	PROJECT TASKS WHICH REQUIRE SOFTWARE TOOL SUPPORT
1	Development of the Mobile App Using Flutter
2	Database Management System
3	Location Filtering
4	Backend Framework
5	Video Player Selection

SOFTWARE TOOLS FOR TASK 1: Mobile Application Framework Selection

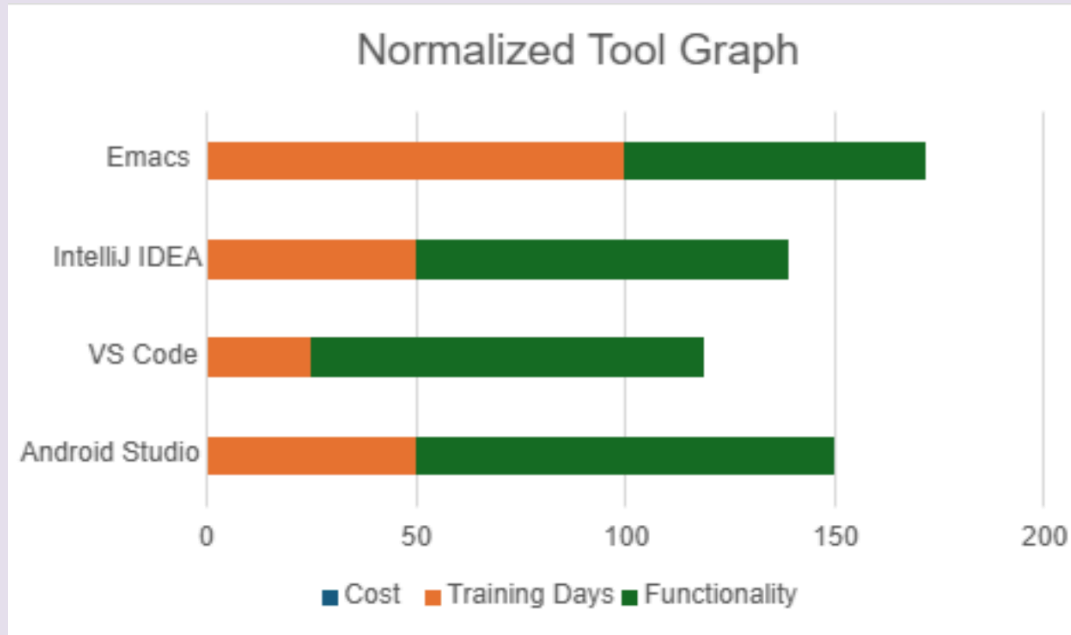
Tool Cost/Training/Functionality Data

Tool	Android Studio	VS Code	IntelliJ IDEA	Emacs	Cursor
Cost	\$0	\$0	\$0	\$0	\$0
Training Days	10	5	10	13	4
Functionality	90	85	80	60	82

Normalized Cost/Training/Functionality Data

Tool	Android Studio	VS Code	IntelliJ IDEA	Emacs	Cursor
Cost	0.0	0.0	0.0	0.0	0.0
Training Days	76.92	38.46	76.92	100.00	30.77
Functionality	100.00	94.44	88.89	66.67	91.11

Normalized Tool Graph



Which tool has been selected? Why?

We chose Android Studio as our primary development environment for Flutter because it offers the most comprehensive and Flutter-specific toolset among all evaluated options. As the official Google-supported IDE for Flutter development, Android Studio provides built-in emulators, advanced debugging tools, and seamless Flutter plugin integration, making it the most suitable choice for our project.

Although VS Code has the shortest learning curve at just 5 days and offers strong Flutter support through extensions, it lacks the depth of built-in mobile development features that Android Studio provides natively. IntelliJ IDEA shares the same core engine as Android Studio but does not include Flutter-specific tools out of the box, requiring additional configuration and resulting in a lower functionality score. Cursor is the easiest to learn at only 4 days and offers powerful AI-assisted code completion, however it is primarily a general-purpose editor and does not provide the dedicated mobile development environment our project requires. Emacs, while highly customizable, has the longest learning curve at 13 days and the lowest functionality score among all options, making it impractical for a time-sensitive Flutter project.

Ultimately, Android Studio's superior functionality score, deep Flutter integration, and strong developer community support make it the most appropriate and reliable choice for developing our mobile application.

SOFTWARE TOOLS FOR TASK 2: Database Management System

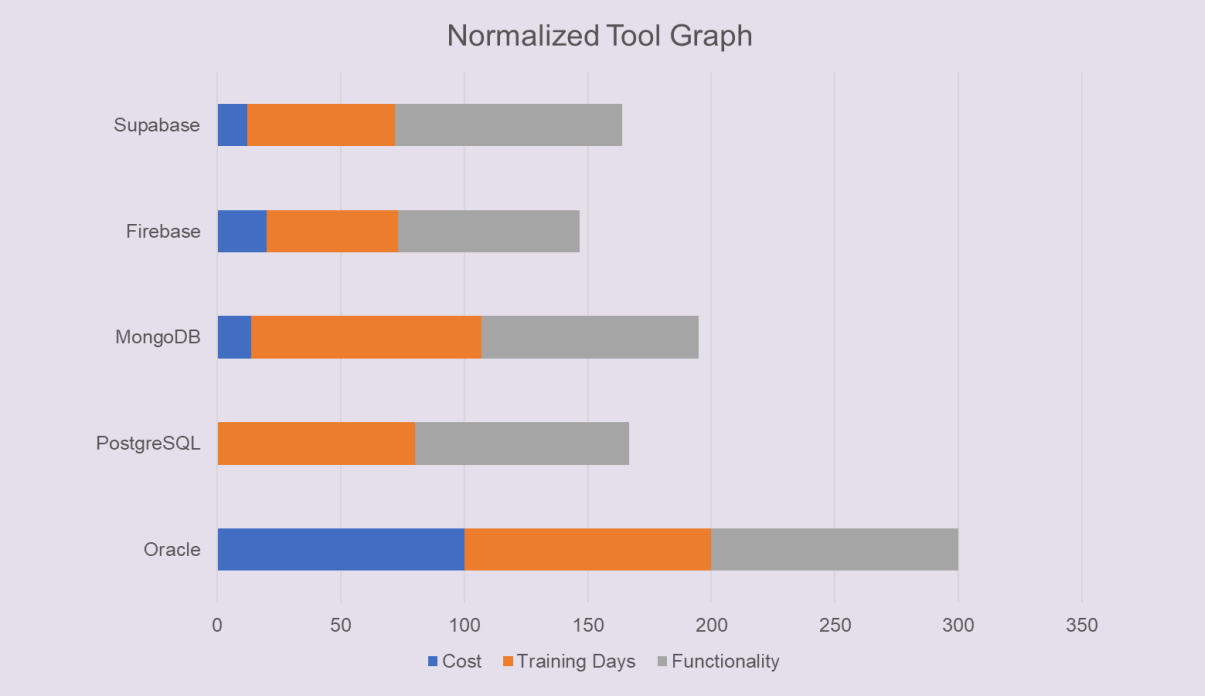
Tool Cost/Training/Functionality Data

Tool	Oracle	PostgreSQL	MongoDB	Firebase	Supabase
Cost	\$5,000/y	\$0	\$684/y	\$1,000/y	\$600/y
Training Days	15	12	14	8	9
Functionality	98	85	86	72	90

Normalized Cost/Training/Functionality Data

Tool	Oracle	PostgreSQL	MongoDB	Firebase	Supabase
Cost	100.0	0.00	13.7	20	12.0
Training Days	100.0	80.0	93.3	53.3	60.0
Functionality	100.0	86.7	87.8	73.5	91.8

Normalized Tool Graph



Which tool has been selected? Why?

As illustrated in the comparison graph above, our selection criteria prioritized establishing a balance between high-level functionality and professional training investment. This balance is particularly important considering that all enterprise database solutions involve significantly high costs. Although PostgreSQL requires a 12-day training period and has a normalized score of 80.0, it stands out as the most sustainable solution because it offers nearly enterprise-level functionality at zero cost, compared to tools such as Supabase at \$600 and Oracle at \$5,000. From the company's perspective, the most cost-effective choice is clearly evident.

This tool possesses the strong relational integrity and advanced security protocols necessary to ensure compliance with KVKK and GDPR, as stated in Non-Functional Requirement 4. Furthermore, it provides the superior data management capabilities required for our AI-based recommendation engine, as specified in Functional Requirement 3, and guarantees high reliability for at least 1,000 concurrent users, as stated in Non-Functional Requirement 9.

Ultimately, PostgreSQL clearly differentiates itself from other database tools through the functional depth and professional stability it provides without imposing any financial burden.

SOFTWARE TOOLS FOR TASK 3: Location Filtering

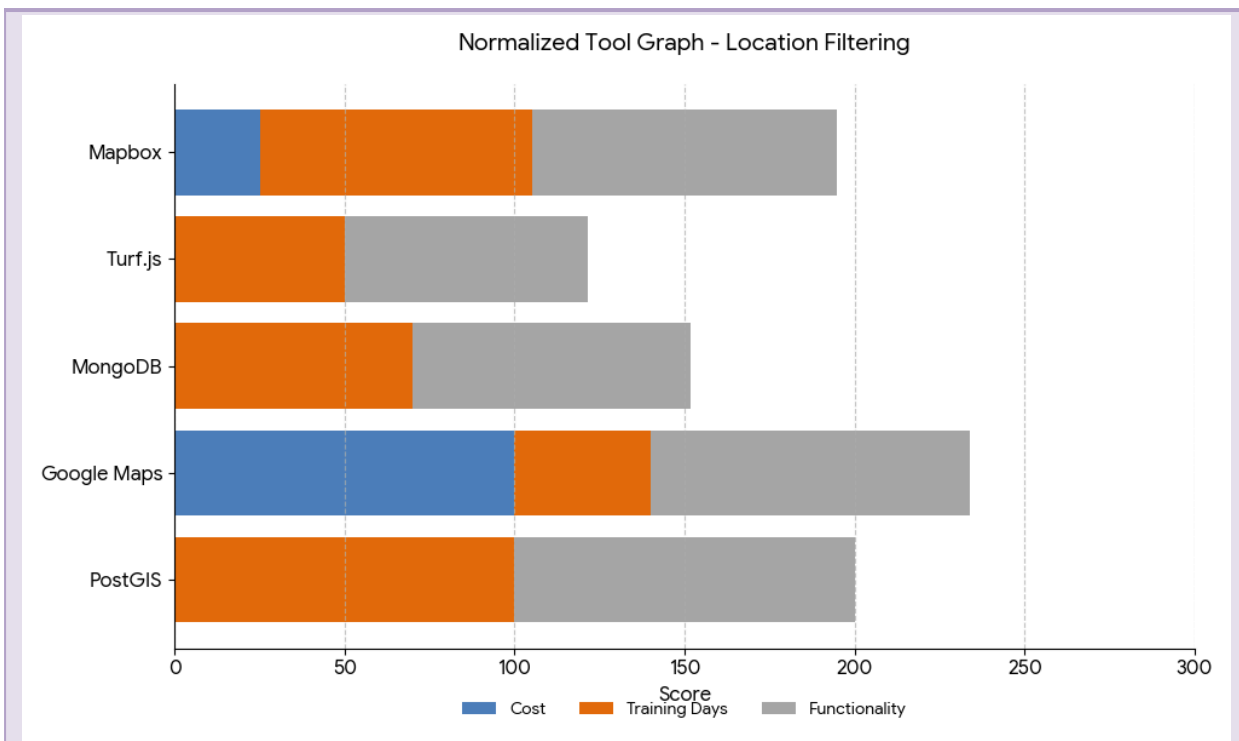
Tool Cost/Training/Functionality Data

Tool	PostGIS	Google Maps API	MongoDB Geospatial	Turf.js	Mapbox SDK
Cost	\$0	\$2,400/y	\$0	\$0	\$600/y
Training Days	10	4	7	5	8
Functionality	98	92	80	70	88

Normalized Cost/Training/Functionality Data

Tool	PostGIS	Google Maps API	MongoDB Geospatial	Turf.js	Mapbox SDK
Cost	0.00	100.00	0.00	0.00	25.00
Training Days	100.00	40.00	70.00	50.00	80.00
Functionality	100.00	93.88	81.63	71.43	89.80

Normalized Tool Graph



Which tool has been selected? Why?

We selected PostGIS as our location filtering tool because it offers the best balance between high-level functionality and zero cost. Since PostgreSQL was already selected for Task 2, PostGIS provides enterprise-level geospatial capabilities without any additional financial burden. This choice ensures strong data management and security protocols necessary for our system requirements. While Google Maps and Mapbox offer high accuracy, they were ruled out due to their high annual costs. Furthermore, MongoDB and Turf.js were excluded to maintain a consistent and powerful relational database structure within our existing NestJS backend.

SOFTWARE TOOLS FOR TASK 4: Backend Framework

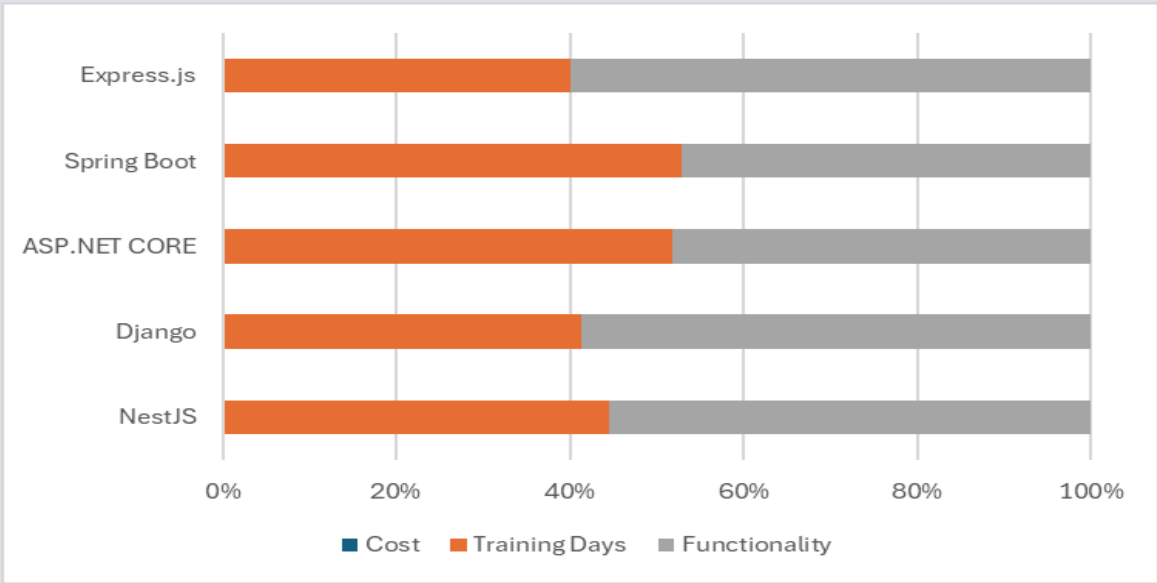
Tool Cost/Training/Functionality Data

Tool	NestJS	Django	ASP.NET CORE	Spring Boot	Express.js
Cost	\$0	\$0	\$0	\$0	\$0
Training Days	12	10	15	15	8
Functionality	95	90	88	85	75

Normalized Cost/Training/Functionality Data

Tool	NestJS	Django	ASP.NET CORE	Spring Boot	Express.js
Cost	0.00	0.00	0.00	0.00	0.00
Training Days	80.0	66.6	100.0	100.0	53.3
Functionality	100.0	94.7	92.6	89.4	78.9

Normalized Tool Graph



Which tool has been selected? Why?

We selected NestJS as our backend framework because it provides the best balance between modularity, functionality, and development speed for our project. Since backend frameworks are open-source, cost was not a deciding factor in our selection, and all options were considered equally in that aspect. NestJS stood out due to its modular structure, which allows us to organize features like user management, job listings, and filtering in a clean and scalable way while enabling team members to work in parallel. Although Django offers fast development and a strong admin panel, it is more monolithic and less flexible for a modular API-based system. ASP.NET Core and Spring Boot are powerful but require more learning time, which is not suitable for our limited schedule. Lastly, Express.js is simple and quick to use but lacks the structured architecture needed for maintaining a larger project. Therefore, NestJS was the most appropriate and balanced choice for our system.

SOFTWARE TOOLS FOR TASK 5: Video Player Selection

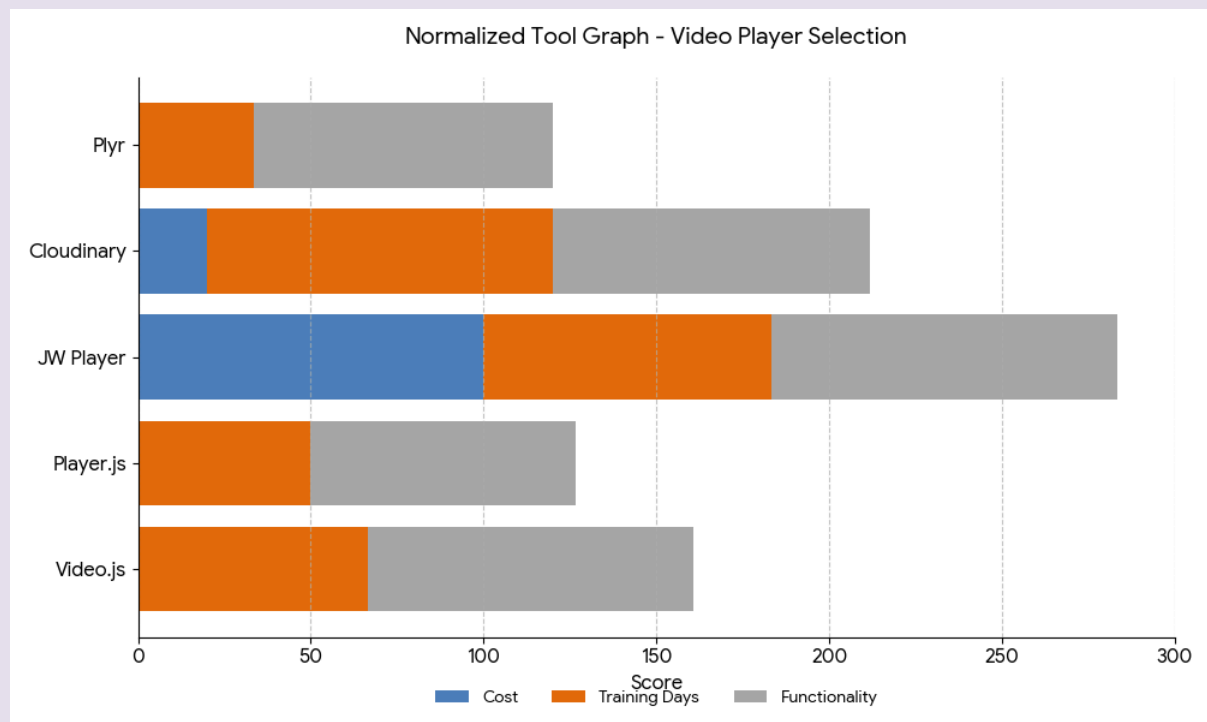
Tool Cost/Training/Functionality Data

Tool	Video.js	Player.js	JW Player	Cloudinary	Plyr
Cost	\$0	\$0	\$6,000/y	\$1,200/y	\$0
Training Days	4	3	5	6	2
Functionality	92	75	98	90	85

Normalized Cost/Training/Functionality Data

Tool	Video.js	Player.js	JW Player	Cloudinary	Plyr
Cost	0.00	0.00	100.00	20.00	0.00
Training Days	66.67	50.00	83.33	100.00	33.33
Functionality	93.88	76.53	100.00	91.84	86.73

Normalized Tool Graph



Which tool has been selected? Why?

We selected Video.js as our primary video player tool because it offers the best balance between extensive functionality and zero cost. Since the framework is open-source, it provides a highly customizable and professional playback environment without any financial burden. It stands out due to its wide range of plugin support and compatibility with various video formats, ensuring a consistent user experience across different devices. While JW Player and Cloudinary offer advanced enterprise features, they were ruled out due to their significantly high annual costs. Furthermore, although Plyr is easier to learn, Video.js was preferred for its superior architectural flexibility and more mature developer community support.